

1. Heuristics

For the **steps** cost measure:

- Pseudocode:

```
for each c1 in currentState and c2 in goal:
    if c1 != c2:
        Add 1 to count
```

- This checks each character between the current state word and the specified goal and add +1 to the cost for each differing letter since there is at least that many actions to take to reach the goal
- The unavoidable remaining cost this lower bounds is the minimum number of steps required to reach the goal (ex. Being able to directly swap letters to the goal while producing dictionary words: dog → dot → cot → cat)
- It cannot overestimate the true cost because it doesn't account for the situations where extra actions must be taken because direct letter swaps towards the goal do not produce a word in the dictionary. (ex. this → that, "thas" and "thit" are not words)

For the **scrabble** cost measure:

- Pseudocode:

```
for each c1 in currentState and c2 in goal:
    if c1 != c2:
        Add c2 scrabble cost to count
```

- This checks each character between the current state and the goal and returns the cumulative scrabble cost of each differing letter in the goal state.
- The unavoidable remaining cost this lower bounds is the summed costs of each letter that can be swapped in legal moves to achieve the goal in the minimum number of steps.
- This cannot overestimate the true cost because any sequence of steps has to contain all the letters in the goal, as well as the cost of each of those letters. Each solution will require paying at least the summed cost of each differing letter from currentState to goal.

For the **frequency** cost measure:

- Pseudocode:

for each c1 in currentState and c2 in goal:

if c1 != c2

Add 1 to count

add goal state rarity to count

- If a mismatched letter is found, add 1 to count. Additionally, add the rarity of the goal word to the count since it can be assumed that the goal state will be reached so the rarity of the goal state word can be considered.
- The unavoidable remaining cost this lower bounds is the minimum number of steps + the word rarity in each step.
- This cannot overestimate the true cost since it does not consider the word rarity of each step, only the goal state since it is guaranteed to reach the goal.

2.

Word Pair	Cost measure	UCS Cost	UCS nodes expanded	A* cost	A* nodes expanded	Reduction
WHY ASK	Steps	5.00	773	5.00	16	97.93%
WHY ASK	Scrabble	9.00	765	9.00	13	98.30%
WHY ASK	Frequency	38.86	808	38.86	755	6.56%
BEAR DUCK	Steps	4.00	691	4.00	6	99.13%
BEAR DUCK	Scrabble	11.00	2981	11.00	6	99.80%
BEAR DUCK	Frequency	38.24	1084	38.24	720	33.58%

- UCS and A* always returned the same optimal cost
- A* produced the largest reduction in expanded nodes for the Scrabble cost for the pair {why → ask} and Scrabble cost for the pair {bear → duck}
- The heuristic was least informative for the Frequency cost measure for both word pairs.
- My steps heuristic estimates based on the number of mismatched letters between the current state and the goal word. My scrabble heuristic improves on

the steps by accounting for the scrabble cost of each mismatched letter in the goal state which allows for the A* search to distinguish paths based on the minimum remaining cost towards the goal state. The frequency heuristic, however, follows similar logic as the steps heuristic, only adding the goal state's rarity value at the end. A* search is unable to optimally decide on the intermediate states because my heuristic doesn't account for the frequency costs of those in-between states from initial/current to the goal state. The frequency cost measure provides less information than the Scrabble and Steps cost measures.

3. Is the optimal cost from W_1 to W_2 equal to the optimal cost from W_2 to W_1 ?

- For the **Steps** cost measure:
 - Yes
 - Because each step only costs 1, the same path from W_1 to W_2 can be followed in reverse while preserving the same optimal cost.
- For the **Scrabble** cost measure:
 - No
 - Since each step costs a specific letter value, following the same path in reverse can yield different costs since the letters that make up W_1 may have different cost values than the letters that make up W_2 .
 - Example:
 - why $\rightarrow$ ask ($a = 1, s = 1, k = 5, C_{\text{minimum}} = 7$)
 - ask $\rightarrow$ why ($w = 4, h = 4, y = 4, C_{\text{minimum}} = 12$)
 - "why" is composed of letters with higher scrabble costs than ask, so at minimum it will cost more to follow the same "optimal path" in reverse
- For the **Frequency** cost measure:
 - No
 - Each step contains a word with a differing rarity value, however following the reverse path will find similar rarities at intermediate steps. However, the different goal states will likely have different rarity values, making the cost differ slightly.
 - Example:
 - why $\rightarrow$ ask (ask rarity: 4.2326436)
 - ask $\rightarrow$ why (why rarity: 3.1326632)
 - The goal state is the only different word in both paths, meaning it is the only one with a different rarity.

4. LLM

- a. I am using ChatGPT-5.6 Luna
 - **Exact Prompt:** In DOGCAT, the initial and goal states are lowercase words of the same length—three or four letters—and every legal state must appear in the supplied dictionary. A legal action changes exactly one letter. Under the frequency objective, entering word w costs $1 + \text{rarity}(w)$, where the nonnegative rarity value is stored in the dictionary. The baseline heuristic $h_0(n)$ is the number of positions in which state n differs from the goal. Propose a rarity-aware heuristic intended to be more informative than h_0 while remaining admissible. Define it precisely and justify your proposal. By “more informative,” we mean that the proposed heuristic should be at least as large as h_0 for every state and strictly larger for at least one state, without overestimating the true optimal remaining cost.
- b. Formula from heuristic: $h_R(n) = \{0, n = g \mid d(n, g) + R(g), n \neq g\}$
 - Where n = current state, g = goal, $d(n, g)$ = number of mismatched letters, and $R(g)$ = rarity of goal state.
- c. The heuristic is admissible.
 - Each step incurs $1 + \text{rarity}(w)$ onto the total cost. The $d(n, g)$ accounts for the 1, but not the $\text{rarity}(w)$ added to the total cost for each step. Every path must also reach the goal at some point, so the additional rarity cost of the goal state must also be considered in the total cost. Each solution path costs some $k + R(g)$, where k = number of mismatched letters in current state, but since the in-between steps aren't accounted for their rarities, $h_R(n) \leq h^*(n)$, where $h^*(n)$ is a true solution path cost. From the informed search lecture slides, $h(n)$ is admissible if $h(n) \leq$ the lowest actual cost from n to goal.
- d. The solution costs and number of expanded nodes of ChatGPT-5.6's heuristic were similar to mine, since both function similarly. I built my frequency heuristic based off of the steps cost measure since each step cost $1 + \text{rarity}(w)$, but I was unable to account for the rarity of each word in the solution path except for the goal state seen in my pseudocode from part 1. The LLM also found it impossible to account for the rarity cost of the next words in the solution path. The evidence here does support using the proposed heuristic, since it has been shown to be admissible and it appears there is no better way of obtaining an accurate path cost due to the inability to know which exact actions may be taken.